

FMC 2026

Software, Auto, Visão e uso de IA na FRC

Sumário

1. Telemetria e logging são requisitos básicos.....	2
2. O auto precisa ser fácil de modificar durante a competição	2
3. Comparar Pose Estimada, Visão e Realidade.....	3
4. PathPlanner precisa ser compreendido, não apenas utilizado	3
Inteligência Artificial	4
5. IA pode acelerar muito o desenvolvimento	4
6. Scouting é uma excelente aplicação para IA.....	4
7. IA no código do robô exige muito mais cuidado	5
8. IA deve ampliar capacidade, não substituir aprendizado	5
9. Alunos mais experientes podem receber maior liberdade.....	6
10. O aluno precisa entender as conexões entre os sistemas	6
11. IA pode introduzir alterações indesejadas	7
Cultura de desenvolvimento	8
12. Logging deve estar presente desde o começo	8
13. Ferramentas precisam servir ao aprendizado	8
Conclusão.....	9
Principal aprendizado	9

1. Telemetria e logging são requisitos básicos

Um dos consensos entre os painelistas foi que **telemetria e logging deixaram de ser opcionais** para equipes que querem trabalhar de forma consistente com auto, visão e diagnóstico.

A principal ideia é simples:

Se o robô apresenta um problema e você não possui logs, você está basicamente tentando adivinhar o que aconteceu.

Em especial para rotinas autônomas, a equipe precisa ser capaz de identificar rapidamente:

- onde o robô acreditava estar;
- onde a câmera acreditava que ele estava;
- qual caminho deveria estar executando;
- quais comandos foram disparados;
- quais sensores estavam ativos;
- quais erros aconteceram durante a partida.

Isso permite corrigir problemas de forma objetiva, em vez de depender apenas da observação visual.

2. O auto precisa ser fácil de modificar durante a competição

Outro ponto importante foi a necessidade de construir rotinas autônomas que sejam **fáceis de ajustar rapidamente**.

Um exemplo citado foi a possibilidade de descobrir, antes de uma partida, que dois robôs da aliança possuem caminhos que se cruzam.

Nesse caso, a equipe precisa conseguir rapidamente:

- adicionar um atraso;
- deslocar um caminho;
- alterar uma trajetória;
- modificar uma pose inicial;
- evitar colisões entre parceiros.

Ou seja, não basta possuir um auto que funciona. A equipe precisa **entender suficientemente bem sua arquitetura para modificá-lo com segurança durante um evento**.

3. Comparar Pose Estimada, Visão e Realidade

Para equipes que utilizam Swerve e visão, os painelistas reforçaram a importância de acompanhar diferentes referências de posicionamento.

Idealmente, a equipe deve conseguir visualizar separadamente:

Pose estimada pelo robô → baseada em odometria, encoders e IMU.

Pose estimada pela visão → baseada nas câmeras e AprilTags.

Posição real observada → onde o robô realmente estava no campo.

Ao comparar essas três informações, fica muito mais fácil identificar se o problema está em:

- odometria;
- visão;
- calibração;
- sistema de coordenadas;
- atraso da câmera;
- integração entre sensores.

Ferramentas como **AdvantageKit/AdvantageScope** foram destacadas justamente por facilitarem essa análise.

4. PathPlanner precisa ser compreendido, não apenas utilizado

Foi comentado que ferramentas como **PathPlanner** facilitam muito o desenvolvimento dos autônomos, mas ainda exigem compreensão por parte dos alunos.

É comum existir diferença entre:

- o que o código deveria executar;
- o que a trajetória planejada representa;
- o que o robô realmente executa.

Por isso, novamente, a telemetria passa a ser fundamental.

Para nossos times, o aprendizado é importante: **utilizar uma biblioteca não elimina a necessidade de entender o que ela está fazendo.**

Inteligência Artificial

5. IA pode acelerar muito o desenvolvimento

Os painelistas apresentaram exemplos reais em que ferramentas como **GitHub Copilot e outras IAs generativas** permitiram desenvolver funcionalidades que anteriormente consumiriam muitas horas.

Foram citados usos como:

- ferramentas internas;
- análise de logs;
- revisão de partidas;
- scripts;
- processamento de dados;
- dashboards;
- aplicações de scouting;
- interfaces para Driver Station.

Em um dos exemplos, um aluno conseguiu desenvolver **quatro novos painéis durante um evento**, utilizando IA como apoio para trabalhar com tecnologias que ainda não dominava completamente.

Isso mostra que a IA pode permitir que uma equipe desenvolva soluções que talvez nem tentasse desenvolver anteriormente por falta de tempo ou conhecimento específico.

6. Scouting é uma excelente aplicação para IA

Um ponto recorrente foi que **scouting e ferramentas auxiliares** são ambientes muito interessantes para utilização de IA.

Aplicações envolvendo:

- JavaScript;
- TypeScript;
- front-end;
- Python;
- análise de dados;
- dashboards;

podem ter uma curva de aprendizado relativamente alta para alunos que estão começando.

A IA pode ajudar o aluno a entrar rapidamente nesses ambientes e desenvolver aplicações funcionais.

Isso é diferente do código crítico do robô, porque uma falha em uma ferramenta de scouting normalmente possui consequências menores do que uma falha no controle de um mecanismo.

7. IA no código do robô exige muito mais cuidado

Os painelistas demonstraram uma postura mais cautelosa em relação ao uso de IA diretamente no **código do robô**.

A preocupação principal não é simplesmente se a IA consegue escrever código.

Ela consegue.

A verdadeira pergunta é:

O aluno entende o código que está utilizando?

Para alguns dos mentores presentes, alunos novos não deveriam utilizar IA para simplesmente gerar o código do robô.

Primeiro, eles precisam desenvolver fundamentos de:

- programação;
- lógica;
- arquitetura;
- comandos;
- subsistemas;
- sensores;
- debugging.

Depois de adquirir essa base, a IA pode ser utilizada como ferramenta de produtividade.

8. IA deve ampliar capacidade, não substituir aprendizado

Essa foi provavelmente a mensagem mais importante da discussão.

O objetivo da FRC não é simplesmente entregar um software funcionando. O programa existe também para que os estudantes **aprendam a desenvolver soluções**.

Portanto, usar IA para fazer uma tarefa que o aluno ainda deveria estar aprendendo pode prejudicar justamente o principal objetivo educacional do projeto.

Um bom uso seria:

“Eu sei aproximadamente como isso funciona, mas não conheço essa biblioteca ou tecnologia. Vou utilizar IA para acelerar meu aprendizado e desenvolvimento.”

Um uso ruim seria:

“Não sei como isso funciona e não quero aprender. Vou pedir para a IA fazer tudo.”

9. Alunos mais experientes podem receber maior liberdade

Uma prática apresentada por uma das equipes foi restringir o uso mais livre de IA aos **alunos mais experientes**.

Isso acontece porque esses estudantes já possuem conhecimento suficiente para:

- identificar erros;
- revisar código;
- questionar respostas incorretas;
- validar arquitetura;
- reconhecer comportamentos inesperados.

Para alunos iniciantes, existe o risco de aceitar qualquer solução gerada simplesmente porque ela compila.

Isso sugere uma abordagem interessante para nossos times:

Novato → aprende fundamentos primeiro.

Intermediário → usa IA com acompanhamento.

Veterano → usa IA como ferramenta de produtividade.

10. O aluno precisa entender as conexões entre os sistemas

Foi citado o caso de um aluno que utilizou IA para desenvolver uma aplicação relacionada à Driver Station.

Os mentores aceitaram o uso, mas exigiram que ele entendesse:

- como os dados chegavam;
- quais interfaces estavam sendo utilizadas;
- como os sistemas se comunicavam;
- o que cada parte do aplicativo estava fazendo.

Ou seja, mesmo que a IA tenha ajudado a escrever parte da aplicação, **o conhecimento da arquitetura continuou sendo responsabilidade do aluno.**

Essa é uma regra muito boa para aplicar:

Você pode utilizar IA, mas precisa ser capaz de explicar o que o código faz e por que ele funciona.

11. IA pode introduzir alterações indesejadas

Um dos painelistas relatou uma experiência com um desenvolvedor que utilizava IA para executar tarefas que não queria fazer manualmente.

O problema é que, ao realizar essas alterações, a IA também modificava elementos que não necessariamente deveriam ser alterados, como constantes e outras partes do projeto.

Isso mostra outro risco importante: uma IA pode resolver a tarefa solicitada e, ao mesmo tempo, introduzir mudanças colaterais.

Portanto, todo código gerado deve passar por:

- revisão;
- comparação de alterações;
- testes;
- validação.

Ferramentas como Git e Pull Requests se tornam ainda mais importantes quando IA é utilizada.

Cultura de desenvolvimento

12. Logging deve estar presente desde o começo

Uma conclusão importante é que logging não deveria ser implementado apenas quando o robô começa a dar problema.

Ele precisa fazer parte da arquitetura desde o início.

Isso permite:

- acompanhar evolução dos mecanismos;
- comparar versões;
- validar tuning;
- investigar falhas;
- analisar partidas;
- comparar comportamento esperado e real.

Para equipes novas, mesmo um logging básico já é muito melhor do que nenhum.

13. Ferramentas precisam servir ao aprendizado

A discussão sobre IA trouxe uma ideia que vale também para bibliotecas e frameworks.

Ferramentas como:

- PathPlanner;
- AdvantageKit;
- Phoenix;
- YAGSL;
- PhotonVision;
- IA generativa;

podem acelerar muito o desenvolvimento.

Mas se forem utilizadas como “caixas pretas”, o aluno pode acabar sem compreender o sistema que está construindo.

A recomendação implícita do painel foi:

Use abstrações, mas entenda as abstrações.

Conclusão

Dessa palestra, eu transformaria o conteúdo em algumas práticas objetivas:

1. **Logging obrigatório em todos os robôs.**
2. Registrar pose, sensores, mecanismos e principais estados do robô.
3. Utilizar AdvantageKit/AdvantageScope ou ferramenta equivalente para análise.
4. Estruturar autônomos para permitir alterações rápidas durante competição.
5. Ensinar os alunos a comparar odometria, visão e posição real.
6. Não utilizar PathPlanner apenas como “gerador de caminho”; entender como ele funciona.
7. Liberar IA inicialmente para ferramentas auxiliares, scouting e análise de dados.
8. Evitar que novatos utilizem IA para gerar diretamente o código principal do robô.
9. Exigir que qualquer aluno consiga explicar o código gerado por IA que esteja utilizando.
10. Utilizar Git e Pull Requests para revisar alterações feitas com IA.
11. Usar IA para acelerar aprendizado, e não para eliminar o processo de aprendizagem.
12. Criar níveis diferentes de autonomia no uso de IA conforme a experiência do aluno.

Principal aprendizado

Essa palestra mostra que a programação FRC moderna está cada vez menos relacionada apenas a **“fazer o motor girar”**.

Equipes de alto desempenho precisam saber **observar o próprio software**. Logging, telemetria, visualização de pose, análise de visão e ferramentas de diagnóstico são o que permitem entender por que um robô funcionou ou não.

Ao mesmo tempo, a chegada da IA cria uma oportunidade enorme de acelerar desenvolvimento, principalmente em ferramentas auxiliares e tecnologias que a equipe ainda não domina. Porém, o princípio educacional precisa continuar claro:

a IA pode escrever código, mas o conhecimento precisa continuar pertencendo ao aluno.